Veritas: Claim Verification & Bias Detection

Design Document
Group 19

Supervisor	
Name: **Email:** **Organization:**	Dr. Allison Lahnala lahnalaa@mcmaster.ca McMaster University, CAS
Team Members	
Name: **Email:** **Student Number:**	Haneef Taher tahers7@mcmaster.ca 400359025
Name: **Email:** **Student Number:**	Keyu Lin link55@mcmaster.ca 400392492
Name: **Email:** **Student Number:**	Madhav Bezawada bezawadm@mcmaster.ca 400380258
Name: **Email:** **Student Number:**	Divij Dhiraaj dhiraajd@mcmaster.ca 400360225
Name: **Email:** **Student Number:**	Louis Doan doanl7@mcmaster.ca 400395183
Name: **Email:** **Student Number:**	Mustakim Kazi kazim18@mcmaster.ca 400390604

1.0 Version History **5**
2.0 Introduction (purpose) **5**
2.1 Project purpose 5
2.2 Document purpose: 5
3.0 Project Component Diagram **6**
4.0 Relationship Between Project Component and Requirements **6**
5.0 Project Components **7**
5.1 UI 7
5.1.1 Normal behaviour: 7
5.1.2 API: 8
5.1.3 Implementation: 8
5.1.4 Potential undesirable behaviour: 8
5.2 Bias Detection 8
5.2.1 Normal behaviour: 8
5.2.2 API and Structure: 8
5.2.3 Implementation: 8
5.2.4 Potential undesirable behaviour: 8
5.3 Bias Detection Training 9
5.3.1 Normal behaviour: 9
5.3.2 API and Structure: 9
5.3.3 Implementation: 9
5.3.4 Potential undesirable behaviour: 9
5.4 Claim Extraction 10
5.4.1 Normal behaviour: 10
5.4.2 API and Structure: 10
5.4.3 Implementation: 10
5.4.4 Potential undesirable behaviour: 10
5.5 Evidence Retrieval 10
5.5.1 Normal behaviour: 10
5.5.3 Implementation: 11
5.5.4 Potential undesirable behaviour: 11
5.6 Claim Veracity Classifier 11
5.6.1 Normal behaviour: 11
5.6.2 API and Structure: 11
5.6.3 Implementation: 12
5.6.4 Potential undesirable behaviour: 12
5.7 Claim Veracity Classifier Training 12

5.7.1 Normal Behaviour:..... 12
5.7.2 API and Structure:..... 12
5.7.3 Implementation:..... 12
5.7.4 Potential undesirable behaviour:..... 13
6.0 User Interface..... 13
7.0 Component Test Plan..... 13
7.1 UI:..... 13
7.1.1 Unit Tests:..... 13
7.1.2 Performance and metrics:..... 13
7.2 Bias Detection Module:..... 14
7.2.1 Unit Tests..... 14
7.2.1.2 Unit Tests Results..... 14
7.2.2 Performance and metrics:..... 14
7.2.2.2 Performance and metrics results:..... 14
7.3 Bias Detection Model Training:..... 15
7.3.1 Performance and metrics..... 15
7.3.1.2 Performance and metrics results..... 15
7.4 Claim Extraction Model:..... 16
7.4.1 Unit Tests..... 16
7.4.2 Performance and Metrics..... 17
7.5 Evidence Retrieval Model:..... 20
7.5.1 Unit Tests..... 20
7.5.2 Performance and Metrics..... 20
7.6 Claim Veracity Module:..... 20
7.6.1 Unit Tests..... 20
7.6.2 Performance and Metrics..... 20
7.7 Claim Veracity Model Training:..... 20
7.7.1 Performance..... 20
7.7.2 Dataset Splitting and Training..... 20
8.0 Component Test Results..... 23
8.5 Evidence Retrieval Model:..... 23
8.5.1 Unit Test Results..... 23
8.5.2 Performance and Metrics Results..... 23
Appendix..... 25
A. Bias Detection Diagram..... 25
B. Bias Detection Training Diagram..... 26
C. Claim Verification Diagram..... 27

D. Current Extension Interface..... 27
E. Figma Link..... 28

1.0 Version History

Version Number	Authors	Description	Date
0	Haneef Taher Keyu Lin Madhav Bezawada Divij Dhiraaj Louis Doan Mustakim Kazi	First Draft	January 23rd, 2026
1	Haneef Taher Keyu Lin Madhav Bezawada Divij Dhiraaj Louis Doan Mustakim Kazi	Align the document to fit closer to the final product	April 7th, 2026

2.0 Introduction (purpose)

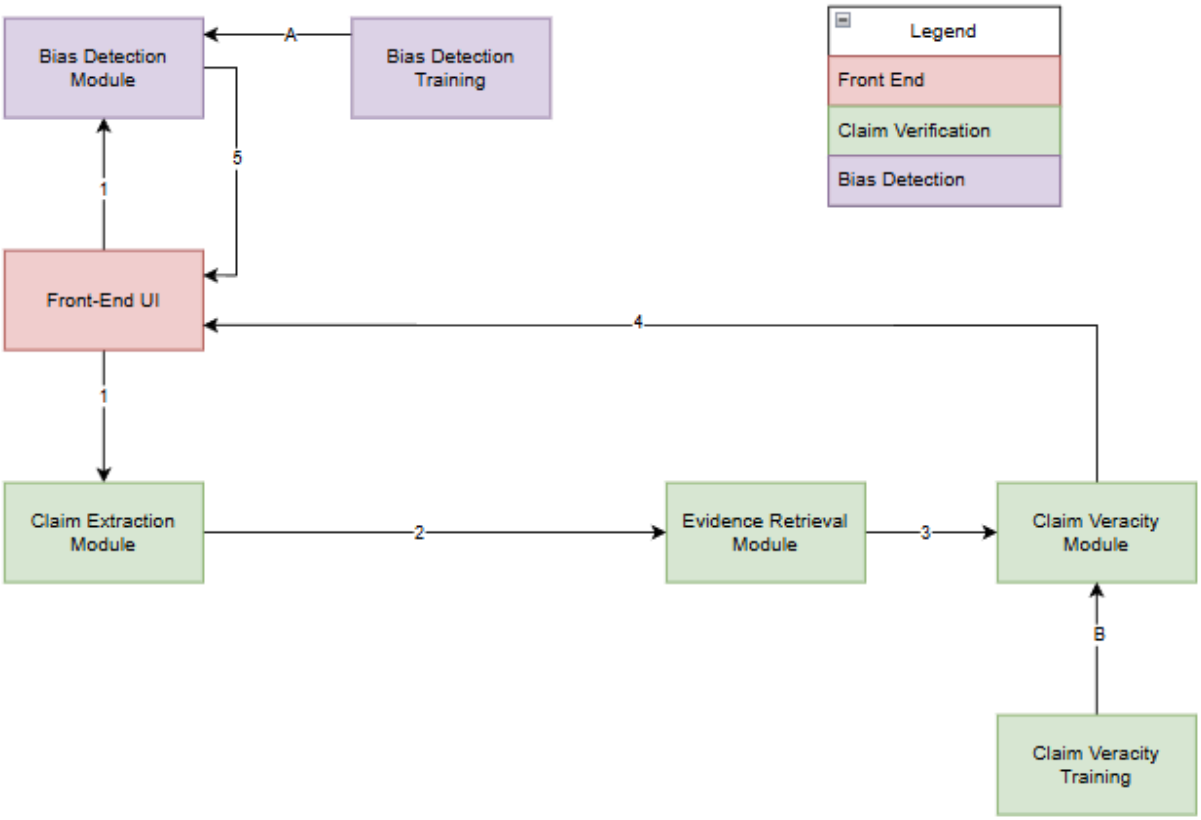
2.1 Project purpose

Misinformation and bias are present in many online spaces, such as news articles, blogs, and forums. The purpose of Veritas is to provide everyday people the ability to quickly and easily analyze online passages of text for factuality and bias. Using our AI models, users will be able to input passages of text they come across online and identify any hidden biases and potential misinformation present. We will then display our findings with supported evidence and sources to the user, keeping them informed about the online content they are consuming.

2.2 Document purpose:

The document outlines various components of our system and provides comprehensive explanations of how they function. It also includes verification and validation of our system requirements to adhere to its goals and robustness. The goal is to provide readers with a high-level understanding of the system's architecture and to show the relationships between its various components.

3.0 Project Component Diagram



1) An article or passage of text is sent from the front end for processing and serviced by the bias detection and claim verification modules
    a) Bias detection module diagram details are in the appendix
2) Text is parsed by an LLM to generate claims and queries for evidence
3) Evidence queries are used to make dynamic web queries or search the static evidence bank to retrieve evidence sentences to use as input to our claim veracity classifier
4) Evaluates the truthfulness of decomposed claims and returns a list of claims, their labels and evidence
5) Bias detection module returns passage and sentence-level bias analysis

A) Bias detection model trained, details in the appendix.
B) Claim veracity model trained, details in the appendix.

4.0 Relationship Between Project Component and Requirements

Project Component	Requirements Covered
Bias Detection	- P0: Analyze the bias of the fully inputted article or an excerpt.

	- P0: Determine the political leaning of the overall content (Left, Center, Right). - P0: Return the results of the bias analysis to the extension for rendering. - Non-functional: Provide confidence scores to the user to support ethical use. - Non-functional: The model generates results in a reasonable amount of time.
Claim Extraction	- P0: Claim verification, for a passage of text, extract claims
Evidence Retrieval	- P0: Claim verification, evidence retrieved from static corpus - P0: Evidence ranked based on relevance to the inputted claim - P1: Cross-reference evidence snippets to reduce hallucinations.
Claim Verification	- P0: Claim verification, with inputted claim and evidence, claim is verified and labelled to be: Supported/Refuted/Not Enough Info
UI	- P0: Create a Chromium browser extension as a front-end for users to interact with our application - P1: Stats dashboard using extension's local history - P2: UI settings with toggles that control counterpoints, evidence, bias and claim verifier - Non-functional: Progress indicators to show the user that some actions are being taken

5.0 Project Components

5.1 UI

5.1.1 Normal behaviour:

The user is able to highlight a passage of text on any webpage and analyze it using the AI model. The results of the analysis will then be displayed to the user through a side panel to the right of the browser.

5.1.2 API:
Selected text from the browser will be sent to the backend through an HTTP request to the API. Backend models will run with the selected text as input and return the output to be displayed within the extension.

5.1.3 Implementation:
The extension is implemented using the WXT framework alongside React and Typescript. The Background.ts file is responsible for retrieving the selected text from the browser and sending it to the backend.

5.1.4 Potential undesirable behaviour:
Text retrieval from the browser may have issues if the user selects problematic elements such as buttons, images, and videos rather than plaintext.

5.2 Bias Detection
5.2.1 Normal behaviour:
The bias detection component is called by the extension when a user chooses to analyze an article or passage of text, and returns a political bias analysis at the sentence level, along with an overall weighted bias leaning for the entire article

5.2.2 API and Structure:
The bias detection model is accessible through the backend API. The API will accept the raw text article/selected passage. Internally, the API functionality will consist of text preprocessing, political content filtering, bias classification, weighted aggregation to compute the overall score of the article, and explainability generation. The output will be this data described as a JSON object for the frontend to interpret and render.

5.2.3 Implementation:
Text preprocessing and sentence splitting are performed using well-known NLP libraries (spaCy) to ensure context is still preserved between sentences. The bias classification model is implemented using a transformer-based architecture, specifically a DeBERTa-v3-large model fine-tuned for political leaning classification. Each sentence is classified with a sliding window of the previous 2 and next 2 neighbouring sentences for context. The output probabilities of the bias classification model are combined using a weighted aggregation method, where quotes and lower-confidence predictions are weighted less. Additionally, we are adding a political filtering classification so only sentences that are political are correctly classified for leaning, and an on-demand explainability feature showing which words in a sentence are most strongly attributed to a biased leaning using gradient x input attribution.

5.2.4 Potential undesirable behaviour:
Potentially undesirable behaviour can reduce accuracy/legitimacy when the input article/passage is heavily sarcastic or otherwise misleading. Additionally, short or cherry-picked passages can result in misleading classifications if sufficient context is not provided. To mitigate

these cases, we encourage users to analyze larger passages within the article or the entire article to produce a more accurate prediction for the bias.

5.3 Bias Detection Training

5.3.1 Normal behaviour:

The model is trained on a compilation of 8 political datasets (article_bias_prediction, dem_rep_party_platform_topics, gpt4_political_bias, gpt4_political_ideologies, political_tweets, qbias, webis_bias_flipper_18, webis_news_bias_20) before being deployed (these datasets are linked in the GitHub readme in the bias-detection folder). Once training, post-hoc deconfounding, and testing are complete, the model is saved for use by the system during production.

5.3.2 API and Structure:

The training/training module consists of several key components. The preprocessing component that prepares the training data and ensures balanced classes, the model initialization that instantiates a pre-trained DeBERTa model, the training component that fine-tunes the model on the training dataset, and the evaluation component that measures how the model performs on the validation and test sets.

5.3.3 Implementation:

For training the model, we combine 8 datasets as listed above to create a unified dataset. This dataset is preprocessed with center-class upsampling to balance class representation. The data is split as follows, 15% set aside for a test set, 100 articles per dataset for validation, and the rest for training (about 60,000).

A transformer-based pre-trained DeBERTa-v3-large model serves as the base, which is fine-tuned using adversarial adaptation with a gradient reversal layer that forces the encoder to learn representations useful for bias classification but unable to reveal which source dataset an article came from. Training is performed using standard supervised learning with cross-entropy loss and runs for 4-12 epochs. Model performance is checked on the validation set to select the best checkpoint.

Once training is complete, LEACE post-hoc source erasure is applied to remove residual source-identifying information from the learned representations. An MLP classifier is then trained on the deconfounded embeddings. The final model is tested on a held-out test set (28,311 samples) and on a recent scrape of 106 AllSides articles to assess generalization and see how it performs on recent data and news trends.

5.3.4 Potential undesirable behaviour:

The model may overfit the training data and accidentally learn associations between specific news sources and their typical political leanings. Additionally, the model may not generalize well to new or evolving political language, as prior work in this domain has shown. To mitigate this, the model is tested on unseen data to assess its generalization capabilities and ensure it does not overfit to the training data.

5.4 Claim Extraction

5.4.1 Normal behaviour:

Claim extraction takes a passage of text, whether an article or a sentence and tries to generate claims extracted from the text using an Mistral 7B self hosted LLM. These extracted claims are passed onto the evidence retrieval module and later claim veracity classification.

5.4.2 API and Structure:

The UI sends a POST request to the claim extraction module. The text is processed and passed onto the next stage of the pipeline, and later receives claim verification output to return as a JSON response to the UI.

5.4.3 Implementation:

FastAPI is used to send an HTTP POST request from the frontend to the claim extraction backend module. The claim extraction module will then make function calls to the LLM to perform text decomposition and claim extraction, returning a list of extracted claims that we can pass to the next step of the pipeline. The pipeline will execute and return final labels, which are passed back up the function calls and bundled into a JSON response to the frontend.

5.4.4 Potential undesirable behaviour:

Users may intentionally or unintentionally submit ill-natured inputs, such as too few words or non-parseable/colloquial text, which may cause processing errors. Alternatively, a passage that doesn't have enough substance to obtain a claim with enough context, can undergoes the claim extraction process, and extract claim(s), wasting resources only to return an invalid output.

5.5 Evidence Retrieval

5.5.1 Normal behaviour:

The evidence retrieval component receives an extracted claim from the claim extraction model. Its expected behaviour is to search a predefined Wikipedia corpus and return a list of relevant evidence passages associated with the input claim. The retrieved evidence is provided as output for use by the claim verification component.

5.5.2 API and Structure:

BM25Retriever(index_path: str):

A retrieval class that wraps a Pyserini BM25 Lucene index over a static Wikipedia corpus. It is responsible for performing an initial lexical search over the corpus.

retrieve(query: str, k: int | None):

Accepts a claim string as input and returns a list of k candidate passages retrieved using BM25. Each returned candidate contains the document identifier, passage text, and BM25 relevance score.

ColBERTReranker(checkpoint: str):

A re-ranking class that applies a ColBERT neural model to re-score BM25 candidates based on semantic similarity to the input claim.

rerank(query: str, candidates: List[dict], top_k: int):
Takes a claim and a list of BM25 candidate passages and returns the top_k ranked passages after ColBERT re-ranking.

has_sufficient_evidence(reranked: List[dict], k: int = 5):
Checks whether the top k ColBERT-ranked passages meet relevance and confidence thresholds required for claim verification.

5.5.3 Implementation:
The implementation follows a two-stage retrieval pipeline:
First, the claim is passed to a BM25Retriever, which queries a pre-built Lucene BM25 index over the Wikipedia corpus and returns a ranked list of n-candidate passages. Each candidate is returned as a dictionary containing the passage docid, BM25 score, and the passage text field used for re-ranking.

Next, the candidate list is passed to ColBERTReranker.rerank(). The reranker builds a temporary "mini-index" over the BM25 candidate texts and runs a ColBERT search using the claim as the query. The re-ranking output is returned as a list of dictionaries containing docid, text, bm25_score, and colbert_score, sorted by ColBERT score and truncated to the final top-k evidence passages. A retrieval confidence check based on ColBERT score margins is applied before verification. low-confidence retrievals are labelled NOT ENOUGH INFO to avoid unreliable verification. For the instances when NOT ENOUGH INFO is determined from the static corpus retrieval, we are currently in the process of implementing real-time web retrieval using different api's (possibly SerpAPI, Bing Web Search API, Brave Search API) to get up-to-date, new relevant evidence.

5.5.4 Potential undesirable behaviour:
Potential undesired behaviours include retrieving irrelevant evidence due to lexical overlap, missing relevant evidence if it is not present in the BM25 candidate set, or increased latency during ColBERT re-ranking. Additionally, ambiguous or poorly formed claims may result in low-quality evidence retrieval, which can negatively impact downstream verification performance.

5.6 Claim Veracity Classifier
5.6.1 Normal behaviour:
Once the claim extraction and evidence retrieval modules have done their work, they should pass on the claim-evidence groups as input into the claim verification module. These inputs are then passed into our model and outputs predicted labels of Supported/Not Enough Info/Refuted which is passed back with the claims and evidence.

5.6.2 API and Structure:
This module will have one access point and main function called(**claim_verification()**) by the evidence retrieval module to do claim classification. Once the generated labels are all processed

with our model, we will return a list of claims-evidence-label back up the module callstack to return to the UI.

5.6.3 Implementation:
The main claim verification function will be called after the evidence retrieval step of the pipeline has finished. Evidence and claim groupings will be passed in as arguments which we can pass into our model to evaluate and obtain labels for. These are joined together and sent back up the callstack as a list structure.

5.6.4 Potential undesirable behaviour:
If the claim lacks enough context due to poor passage selection by the user, the evidence retrieval process is rendered redundant and the classifier will simply return not enough evidence. There might also be cases of cherrypicked statistical data that motivates our classifier to present a psuedo-conclusive result. Inaccuracies such as false positives and false negatives can be very harmful as they inform the user with "evidence" promoting no further evaluation of the passage material.

5.7 Claim Veracity Classifier Training
5.7.1 Normal Behaviour:
Training for the claim verification model is done with datasets such as Averitec and FEVER before the application is deployed. The trained model is saved for loading during production.

5.7.2 API and Structure:
We can choose to train on a subset of the datasets, whether that's only selecting one dataset or a partition of one or both datasets and observe the effects on model performance. For production and final deployment, we will train on all available data(provided it results in the best performance). In training_order.py, we run this file to execute a training order on the selected datasets in the main() function.

5.7.3 Implementation:
To train the claim verification model, annotated claim–evidence pairs from benchmark datasets such as FEVER and AveriTeC are first prepared for supervised learning. Each data instance consists of a natural language claim, one or more evidence sentences, and a corresponding verification label (supported, refuted, or not enough information).

We use a multi-evidence architecture that includes a claim-aware attention mechanism. In this setup, each claim is processed alongside all of its evidence sentences simultaneously. The model computes attention weights over each evidence sentence based on its relevance to the claim and aggregates the evidence into a weighted representation. This architecture enables the model to learn which sentences contribute most to the final verdict, supporting better reasoning and transparency.

For training, the combined dataset consisting of FEVER and AveriTeC examples is used. The model is trained from scratch or fine-tuned on FEVER first and then AveriTeC, depending on the

experiment. Data is loaded using a custom dataset interface that batches examples and shuffles them for training stability. If a previously trained checkpoint is available, it is loaded to resume training.

The training procedure runs for a fixed number of epochs. During each iteration, the model predicts labels for a mini-batch, computes the loss using cross-entropy with respect to ground-truth annotations, and updates weights using gradient descent. The final model checkpoint is saved for evaluation and deployment.

5.7.4 Potential undesirable behaviour:
As with all ML models, we can succumb to pitfalls such as underfitting, overfitting, bad data or good performance metrics as red herrings. It could also learn bad or improper relations between claim, evidence, and label, which would be costly to diagnose and fix. Retraining this model would be very time and resource consuming.

6.0 User Interface

Our extension adds a side panel on the right of the browser as users navigate between various webpages. It is designed to be non-intrusive, not getting in the way of users as they browse the internet, while presenting users with easy-to-understand information at a glance.
When users want to inspect a passage of text for hidden biases and fact-checking, they can effortlessly highlight the text, right-click, and click “Analyze text”. This brings up the side panel along with the detailed analysis results generated from Veritas AI.
To provide further ease of use, analyzed passages are directly underlined on the webpage. When an analysis item is hovered over in the side panel, the corresponding passage on the actual webpage will be scrolled to, and vice versa. This bidirectional link allows users to easily understand which piece of analysis pertains to each specific passage of text.
The usage of colour is also used to help users’ understanding. Specific results correspond to specific colours as can be seen below. Together, all of this gives users the information they want at a glance. See Figure D in the appendix for the current implementation, and Figure E for the Figma design link.

7.0 Component Test Plan

7.1 UI:

7.1.1 Unit Tests:
We will be using Vitest to test components of the extension where applicable. Common failure cases, such as API or authentication errors, should be handled gracefully and displayed to the user. We expect the process of sending and receiving data from the backend to be reliable with any errors being clearly communicated to the user.

7.1.2 Performance and metrics:
We expect the process of sending the selected text from the frontend extension to the backend AI model and receiving the corresponding output to complete within 20 seconds. This is

dependent on the length of the input, but we expect the system to scale reasonably, especially for shorter passages of text.

7.2 Bias Detection Module:

7.2.1 Unit Tests

Our core testing strategy combines functional validation using passages from political news articles across AllSides, edge case handling including empty inputs, whitespace-only inputs, gibberish content that our politicalness classifier should reject, single-word inputs, and non-political text (e.g. recipes, sports). We also validate response structure (correct JSON fields, valid probability distributions, confidence score ranges, valid category labels), explainability endpoint behaviour (token attribution scores, stopword filtering), and latency benchmarks on real news articles of varying lengths.

7.2.1.2 Unit Tests Results

We ran a total of 16 tests, and all 16 tests passed. The tests (found in the bias-detection/backend folder in the test_bias_api.py file) encompass 7 edge case tests for health, empty inputs, whitespace only, missing JSON, gibberish is not classified as political, single words, and recipes. 5 tests for the actual bias model encompassing a valid bias prediction with probabilities summing to 1, response contains all the correct fields, testing confidence score range, category labels, issue count can't exceed check count. 3 explainability tests to ensure it returns the right amount of tokens with scores, empty input, stopwords get rejected. Finally, a latency test on various length articles to ensure the model can produce an output in a reasonable amount of time (see section 7.2.2.2).

7.2.2 Performance and metrics:

The bias detection module is subject to the time constraints where low end-to-end latency is critical for a responsive user experience. Runtime performance will be evaluated using real news articles of varying lengths. The target runtime is under 15 seconds per request on average, ensuring that users do not experience noticeable delays, though we expect short sentences and paragraphs to be much quicker.

7.2.2.2 Performance and metrics results:

Tests were performed on GPU using 3 real articles of various length:

Length	Time
~150 words	2.4s
~450 words	2.7s
~750 words	3.0s
Average	2.7s

7.3 Bias Detection Model Training:

7.3.1 Performance and metrics

The model will be evaluated using standard classification metrics, including precision, recall, and F1 score, to assess its performance. Additionally, we will use confusion matrices to identify misclassification patterns (some models tend to overclassify certain political leanings) and ensure our model is accurate and fair across all three classes.

To reduce overfitting the data is split as follows: 15% per dataset is held out as a test set, 100 samples per dataset are reserved for validation, and the remainder is used for training (roughly 60,000 samples). Furthermore, the model will be tested on newly scraped AllSides data to assess its performance on current trends and how well it generalizes to out-of-distribution data.

7.3.1.2 Performance and metrics results

Overall results:

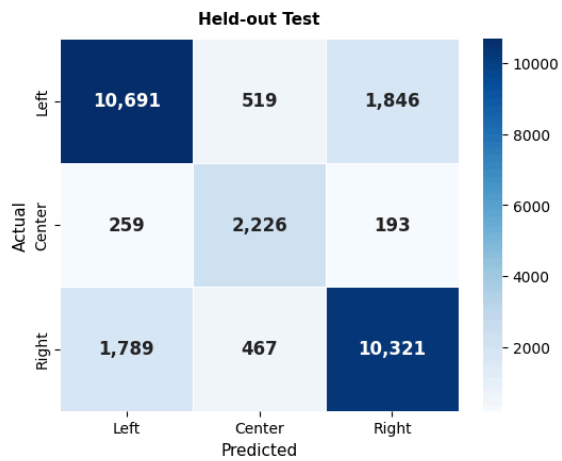
Split	Accuracy	Macro F1
Validation set (in-distribution)	86.1%	0.849
Test set (in-distribution)	82.1%	0.804
AllSides OOD (out-of-distribution)	61.3%	0.612

The model exceeds the SRS target of ~0.75 macro F1 for in-distribution. Additionally, the AllSides OOD test the model achieves an accuracy of 61.3%, which is a roughly 14% increase over the Volf et al. baseline, which got 47.2%.

Per-class results on test set:

Class	Precision	Recall	F1
Left	0.84	0.82	0.83
Center	0.69	0.83	0.76
Right	0.84	0.82	0.83

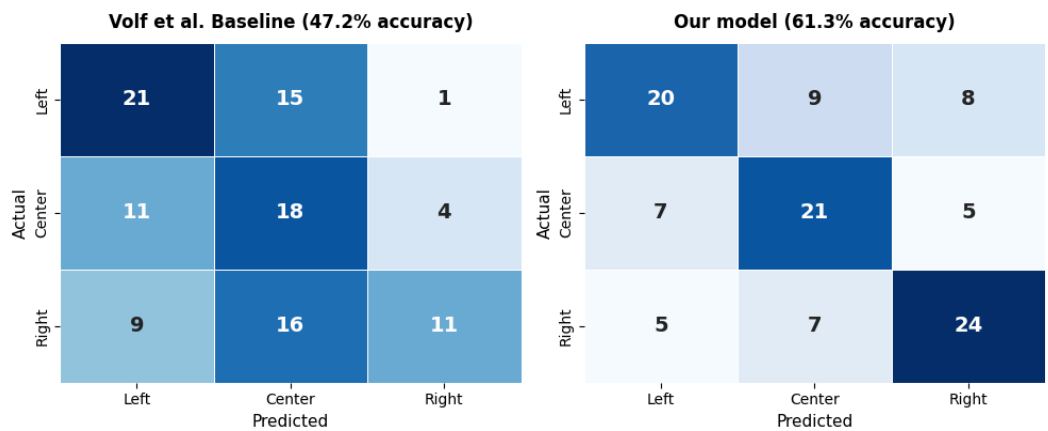
Confusion matrix on test set:



Per-class results on AllSides OOD:

Class	Precision	Recall	F1
Left	0.62	0.54	0.58
Center	0.57	0.64	0.60
Right	0.65	0.67	0.66

Confusion matrix on Allsides OOD (vs. the baseline):



7.4 Claim Extraction Model:

7.4.1 Unit Tests

For our claim extraction module, we aim to test the robustness and correct behaviour of claim extraction functionality. We can do this with a mixture of boundary and partition tests and test for cases such as empty strings, short strings, long passages. Our mocked LLM should be able to return a correctly formatted list of claims. Our unit tests should also test behaviour such as response JSON being correctly formatted and returned as contracted. Unit tests in Python can be implemented with the unittest library.

All test cases for claim extraction are contained tests/CE_tests/test_claim_extraction.py and can all pass.

7.4.2 Performance and Metrics

We aim to keep claim extraction processing time low as there are multiple steps to the claim verification pipeline to ensure a smooth experience. We aim to keep mean response time of claim extraction to under 10 seconds. As passage length varies, response times can vary as well.

In addition to our performance and latency requirements, we should be able to benchmark and evaluate our claim extraction model. We do this by taking our LLM model(in our demo and expo case, Mistral) and evaluating against a self made gold standard dataset that contains text passages and reference claim. Later on we test against the FEVERFact dataset which also contains main text passages, reference claims, and context sentences which are meant to have claims extracted on but given context to claims extracted from the main sentences. We also test with zero-shot and one-shot prompts with zero-shot later serving as a baseline. The evaluations on these datasets include metrics such as semantic precision, semantic recall, semantic F1 which indicate how close extracted claims are to reference claims(precision) and how many of the reference claims were extracted(recall).

Our method of evaluating if an extracted claim matches a reference claim uses an all-MiniLM-L6-v2 model to calculate cosine similarity as a measure of semantic similarity. If it scores above the similarity threshold of 0.8, then it's considered a matched extracted claim. We can set our semantic F1 goal as > 0.72 on the self-gold dataset and >0.37 for the FEVERFact dataset. These thresholds were all chosen ad-hoc as there isn't much literature on these benchmarks and topic and I used zero-shot outputs as a guideline for a baseline.

Note that these benchmarks were taken on a system with Ryzen 7 4700U, 32GB of RAM, Nvidia 4060Ti GPU.

Results

Gold/Self Dataset + Zero Shot:

	mistral	mixtral	llama3	Phi3	gemma	deepseek-r1
Avg Semantic Precision	0.819	DNC	0.915	0.815	0.846	DNC
Avg Semantic Recall	0.659	DNC	0.904	0.682	0.784	DNC
Avg Semantic F1	0.722	DNC	0.905	0.730	0.891	DNC
Avg Latency	2.05s	DNC	6.77	1.89	11.43	DNC

Max Latency	3s	DNC	11.12	4.17	19.59	DNC

Mixtral consumes a lot of memory that we couldn't feasibly run locally. Deepseek had major variability and response time issues which never terminated.

Baseline Gold/Self Dataset + Zero Shot(mistral):

```
=====================================================
  Dataset            : Gold annotations (gold_annotations.json, n=8)
  Model              : mistral
  Similarity threshold: 0.8
  Passages evaluated : 8  (0 errors)
-----------------------------------------------------
  Avg Semantic Precision : 0.833
  Avg Semantic Recall    : 0.871  (target ≥ 0.75)
  Avg Semantic F1        : 0.845
  Avg latency            : 3.47s  (target < 10s)
  Max latency            : 4.82s
=====================================================
  ✓ Recall target (≥ 0.75) met.
  ✓ Latency target (< 10s) met.

Full results saved to api\claim_extraction\eval\results_gold.json
```

Baseline FEVER Dataset + Zero Shot(mistral):

```
=====================================================
  Dataset            : FEVERFact (feverfact.jsonl, n=444 )
  Model              : mistral
  Similarity threshold: 0.8
  Passages evaluated : 444  (0 errors)
-----------------------------------------------------
  Avg Semantic Precision : 0.315
  Avg Semantic Recall    : 0.528  (target ≥ 0.75)
  Avg Semantic F1        : 0.376
  Avg latency            : 2.84s  (target < 10s)
  Max latency            : 8.88s
=====================================================
  ⚠Recall 0.528 is below the 0.75 target.
  ✓ Latency target (< 10s) met.
```

FEVER 50 samples + One Shot:

	mistral	llama3	Phi3	gemma
Avg Semantic Precision	0.277	0.272	0.277	0.261
Avg Semantic Recall	0.590	0.578	0.532	0.513
Avg Semantic F1	0.358	0.346	0.348	0.331
Avg Latency	4.59	6.49	2.42	16.86
Max Latency	7.69	14.15	5.59	37.49

Taking out the models that didn't finish claim extraction on the gold dataset, gemma exhibits way too long of a response time.

Full FEVER Dataset + One Shot:

```
=====================================================
  Dataset            : FEVERFact (feverfact.jsonl, n=444 )
  Model              : mistral
  Similarity threshold: 0.8
  Passages evaluated : 444  (0 errors)
-----------------------------------------------------
  Avg Semantic Precision : 0.340
  Avg Semantic Recall    : 0.631  (target ≥ 0.75)
  Avg Semantic F1        : 0.421
  Avg latency            : 4.95s  (target < 10s)
  Max latency            : 12.43s
=====================================================
  ⚠Recall 0.631 is below the 0.75 target.
  ⚠Max latency 12.43s exceeds the 10s target.

Full results saved to api\claim_extraction\eval\results_feverfact.json
```

	mistral	llama3	Phi3
Avg Semantic Precision	0.360	0.306	0.282
Avg Semantic Recall	0.631	0.585	0.484
Avg Semantic F1	0.421	0.380	0.335
Avg Latency	4.95	6.75	2.71
Max Latency	12.43	19.18	6.92

Mistral slightly outperforms llama3 while also being faster. Phi3 performs worse than both the other models but this is expected of a lightweight model, reflected in its much lower latency times.

We can see that our one-shot methodology for extracting claims performs better than zero-shot which serves a baseline. There is a noticeable drop between the self-gold and FEVER dataset but this is due to the structure FEVERFact dataset containing context sentences that had claims also extracted on them which drastically decreased precision. If we only extract on main sentences, we miss context included in context sentences and our extracted claims don't hit the similarity threshold and suffer in recall. In a real use context, we expect to receive more textual context and extract claims on the entire passage unlike the FEVERFact dataset so it's a safe assumption that the performance and metrics on real passages would be better.

Out of all the models to use, mistral is the best choice for our needs of a lightweight fast model while retaining good results. Phi3 might be best suited in cases where we need faster performance such as demoing off a lower specification laptop.

7.5 Evidence Retrieval Model:

7.5.1 Unit Tests

Unit tests for evidence retrieval include verifying that extracted claims consistently result in a properly formatted list of evidence passages, where each entry contains the expected fields such as passage text and associated identifiers or relevance scores. The retrieval pipeline should return a non-empty list when relevant evidence exists, handles edge cases such as empty, very short, or ambiguous claims without failure, and produces an empty or partial result in a controlled manner when evidence cannot be found. Additional tests will ensure that both the BM25 retrieval stage and the ColBERT re-ranking stage return outputs in the expected format and that the final evidence list is correctly structured and usable by the claim verification component.

7.5.2 Performance and Metrics

Evidence retrieval will likely be the biggest processing bottleneck in our claim verification pipeline so we will monitor latency performance closely for ways to improve. We aim to finish evidence retrieval within 30 seconds. We can also observe latency impact given size of text and map it out to see its relation as part of performance analysis.

Evidence Retrieval Results

The Evidence Retrieval component was tested using automated unit tests covering the BM25 retrieval stage, ColBERT re-ranking stage, confidence logic, and the full pipeline. These tests verified correct output structure, proper sorting of results, and safe handling of empty or malformed inputs. End-to-end tests ensured that the pipeline returns the expected format, handles empty claims gracefully, respects the configured number of evidence passages, and triggers live-news fallback when local evidence is insufficient. Confidence tests validated behavior under different score conditions, including empty results, identical scores, and clear top matches. The test suite was run using pytest, and all 15 tests passed, as shown below. This confirms that the retrieval component behaves correctly under the tested scenarios and produces outputs suitable for the claim verification stage.

```
============================= test session starts ==============================
platform linux -- Python 3.12.12, pytest-9.0.2, pluggy-1.6.0 -- /usr/bin/python
cachedir: .pytest_cache
rootdir: /home/bezawadm/TL69---Capstone
plugins: anyio-4.13.0
collecting ... collected 15 items

tests/test_er_bm25.py::test_bm25_retrieve_returns_expected_fields PASSED [  6%]
tests/test_er_bm25.py::test_bm25_retrieve_handles_invalid_raw_json PASSED [ 13%]
tests/test_er_bm25.py::test_bm25_retrieve_uses_default_k_when_not_overridden PASSED [ 20%]
tests/test_er_colbert.py::test_colbert_rerank_returns_sorted_expected_fields PASSED [ 26%]
tests/test_er_colbert.py::test_colbert_rerank_skips_empty_candidates PASSED [ 33%]
tests/test_er_confidence.py::test_confidence_returns_false_for_empty_results PASSED [ 40%]
tests/test_er_confidence.py::test_confidence_returns_true_for_single_result PASSED [ 46%]
tests/test_er_confidence.py::test_confidence_returns_false_when_scores_are_identical PASSED [ 53%]
tests/test_er_confidence.py::test_confidence_returns_false_when_top_gap_is_too_small PASSED [ 60%]
tests/test_er_confidence.py::test_confidence_returns_true_for_clear_top_hit PASSED [ 66%]
tests/test_er_pipeline.py::test_retrieve_returns_expected_contract PASSED [ 73%]
tests/test_er_pipeline.py::test_retrieve_handles_empty_results PASSED    [ 80%]
tests/test_er_pipeline.py::test_retrieve_uses_live_news_fallback_when_local_evidence_is_not_confident PASSED [ 86%]
tests/test_er_pipeline.py::test_retrieve_respects_evidence_k PASSED      [ 93%]
tests/test_er_pipeline.py::test_latency_budget_with_stubbed_components PASSED [100%]
```

A latency benchmark was run on 25 claims using the evidence retrieval pipeline, with each claim returning 3 evidence passages. The benchmark measured retrieval time along with evidence count, confidence, and source type.

On a CPU-based setup, the mean latency was 3.43 seconds, the median was 3.48 seconds, and the maximum was 4.31 seconds, with all claims completing within the 30-second target.

On the McMaster GPU servers, performance improved significantly, with a mean latency of 0.17 seconds, a median of 0.15 seconds, and a maximum of 0.51 seconds, again with all claims within the 30-second budget.

These results show that the retrieval component meets the latency requirement under both configurations, with GPU acceleration providing a substantial speedup and making the system suitable for real-time use.

CPU Latency Test

```
[1/25] 3.00s | evidence=3 | source=local
[2/25] 4.17s | evidence=3 | source=local
[3/25] 3.78s | evidence=3 | source=local
[4/25] 3.36s | evidence=3 | source=local
[5/25] 3.70s | evidence=3 | source=local
[6/25] 2.75s | evidence=3 | source=local
[7/25] 2.82s | evidence=3 | source=local
[8/25] 3.39s | evidence=3 | source=local
[9/25] 3.48s | evidence=3 | source=local
[10/25] 3.69s | evidence=3 | source=local
[11/25] 3.25s | evidence=3 | source=local
[12/25] 2.92s | evidence=3 | source=local
[13/25] 3.76s | evidence=3 | source=local
[14/25] 2.95s | evidence=3 | source=local
[15/25] 2.93s | evidence=3 | source=local
[16/25] 3.09s | evidence=3 | source=local
[17/25] 2.92s | evidence=3 | source=local
[18/25] 3.66s | evidence=3 | source=local
[19/25] 3.93s | evidence=3 | source=local
[20/25] 3.58s | evidence=3 | source=local
[21/25] 3.94s | evidence=3 | source=local
[22/25] 3.27s | evidence=3 | source=local
[23/25] 3.57s | evidence=3 | source=local
[24/25] 4.31s | evidence=3 | source=local
[25/25] 3.57s | evidence=3 | source=local

ER latency summary
claims tested: 25
mean latency: 3.43s
median latency: 3.48s
max latency: 4.31s
within 30s budget: 25/25
```

GPU Latency Test

```
[1/25] 0.51s | evidence=3 | source=local
[2/25] 0.20s | evidence=3 | source=local
[3/25] 0.18s | evidence=3 | source=local
[4/25] 0.14s | evidence=3 | source=local
[5/25] 0.15s | evidence=3 | source=local
[6/25] 0.12s | evidence=3 | source=local
[7/25] 0.13s | evidence=3 | source=local
[8/25] 0.17s | evidence=3 | source=local
[9/25] 0.17s | evidence=3 | source=local
[10/25] 0.17s | evidence=3 | source=local
[11/25] 0.14s | evidence=3 | source=local
[12/25] 0.13s | evidence=3 | source=local
[13/25] 0.18s | evidence=3 | source=local
[14/25] 0.13s | evidence=3 | source=local
[15/25] 0.13s | evidence=3 | source=local
[16/25] 0.19s | evidence=3 | source=local
[17/25] 0.13s | evidence=3 | source=local
[18/25] 0.17s | evidence=3 | source=local
[19/25] 0.15s | evidence=3 | source=local
[20/25] 0.12s | evidence=3 | source=local
[21/25] 0.15s | evidence=3 | source=local
[22/25] 0.14s | evidence=3 | source=local
[23/25] 0.17s | evidence=3 | source=local
[24/25] 0.18s | evidence=3 | source=local
[25/25] 0.15s | evidence=3 | source=local

ER latency summary
claims tested: 25
mean latency: 0.17s
median latency: 0.15s
max latency: 0.51s
within 30s budget: 25/25
```

7.6 Claim Veracity Module:

7.6.1 Unit Tests

We added a small regression test suite in tests/CV_tests/test_claim_veracity_regression.py that covers the three output classes: SUPPORTED, REFUTED, and NOT ENOUGH INFO. Each test uses a fixed claim + evidence pair and checks that (1) the output label is valid, (2) probabilities include all labels, and (3) the expected label matches. These tests are lightweight and ensure our claim-verification pipeline and label mapping stay stable across code or model updates.

7.6.2 Performance and Metrics

We measured the evaluation time of the claim veracity pipeline on the AveriTeC dataset. We run the official evaluation script and report the wall-clock runtime to be around 4 seconds for 182

claims using an NVIDIA H100 GPU. This provides a baseline for model throughput and helps detect performance regressions when models or preprocessing are updated.

7.7 Claim Veracity Model Training:

7.7.1 Performance

Model training is completed offline before deployment. The live system loads a pre-trained, fine-tuned checkpoint. Our training runs are short by design to allow for iteration, and in practice have taken under one hour when training on any of the McMaster GPU servers.

7.7.2 Dataset Splitting and Training

We are splitting our dataset 60/20/20 to have a fair evaluation of our model metrics. 60% of the data is used to train, 20% as the validation set and the remaining 20% is used to evaluate and calculate various metrics we will further discuss below. We implement early stopping to prevent overfitting and end training when our validation progress converges.

7.7.3 *Results*

We compare our latest model which was trained on both FEVER and averitec to act as a single inference model to baselines that were trained and evaluated on the same domain. We show that our model can perform consistently across both domains.

Evaluation metrics of shared multi head attention model on fever:

	precision	recall	f1-score	support
REFUTED	0.74	0.62	0.68	6085
NOT ENOUGH INFO	0.63	0.45	0.52	6999
SUPPORTED	0.75	0.89	0.81	16006
accuracy			0.73	29090
macro avg	0.70	0.65	0.67	29090
weighted avg	0.72	0.73	0.71	29090

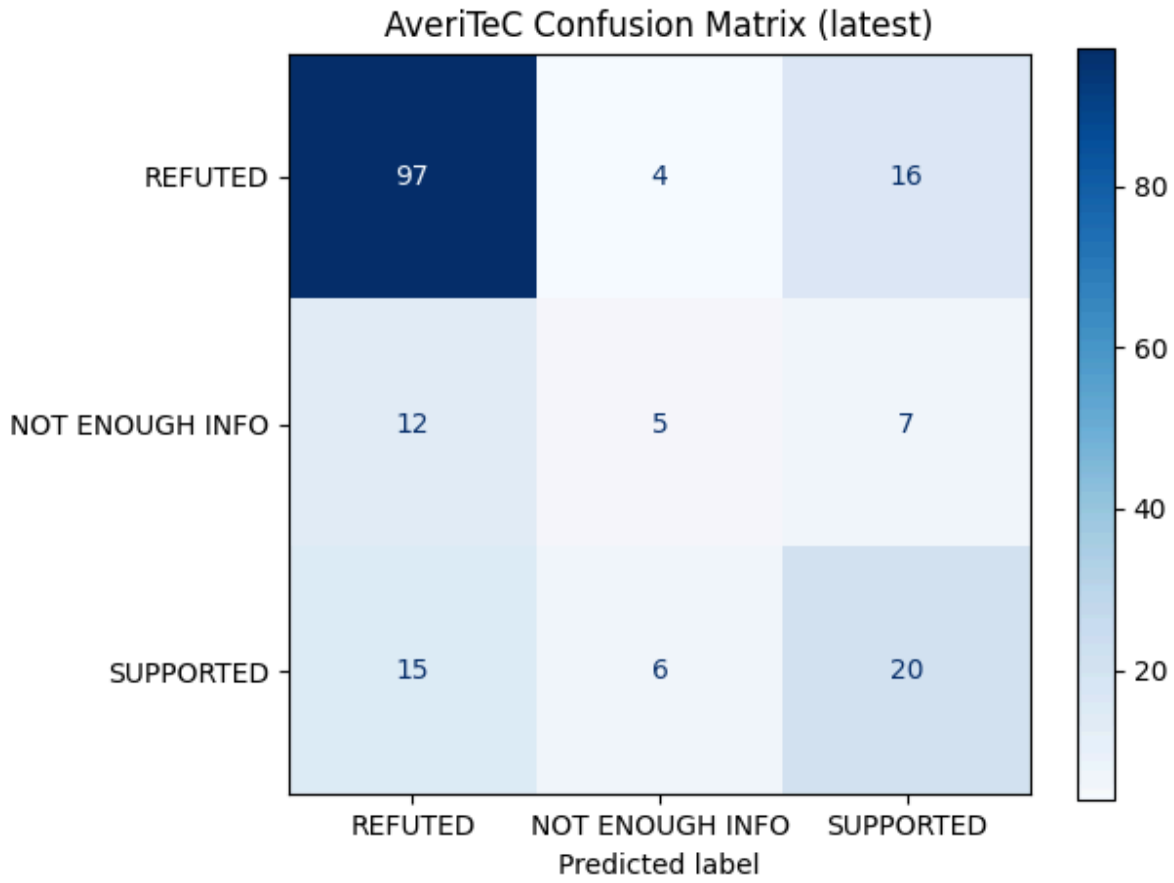
We achieved a FEVER score of 0.58 and the evidence retrieval model had a hit rate of 0.75 on the non-NEI evidence claims, which is also how the official fever score evaluates the retriever as it disregards evidence attached to NEI claims.

Performance is strongest on SUPPORTED (F1 0.81, recall 0.89) labels and weakest on NOT ENOUGH INFO (F1 0.52) labels due to the class imbalance.

Model	Accuracy	Macro F1
Shared multi head attention model (with retrieved evidence)	0.73	0.67
Baseline model (trained and evaluated on Fever)	0.71	0.63

Evaluation metrics of shared multi head attention model on averitec:

	precision	recall	f1-score	support
REFUTED	0.78	0.83	0.80	117
NOT ENOUGH INFO	0.33	0.21	0.26	24
SUPPORTED	0.47	0.49	0.48	41
accuracy			0.67	182
macro avg	0.53	0.51	0.51	182
weighted avg	0.65	0.67	0.66	182

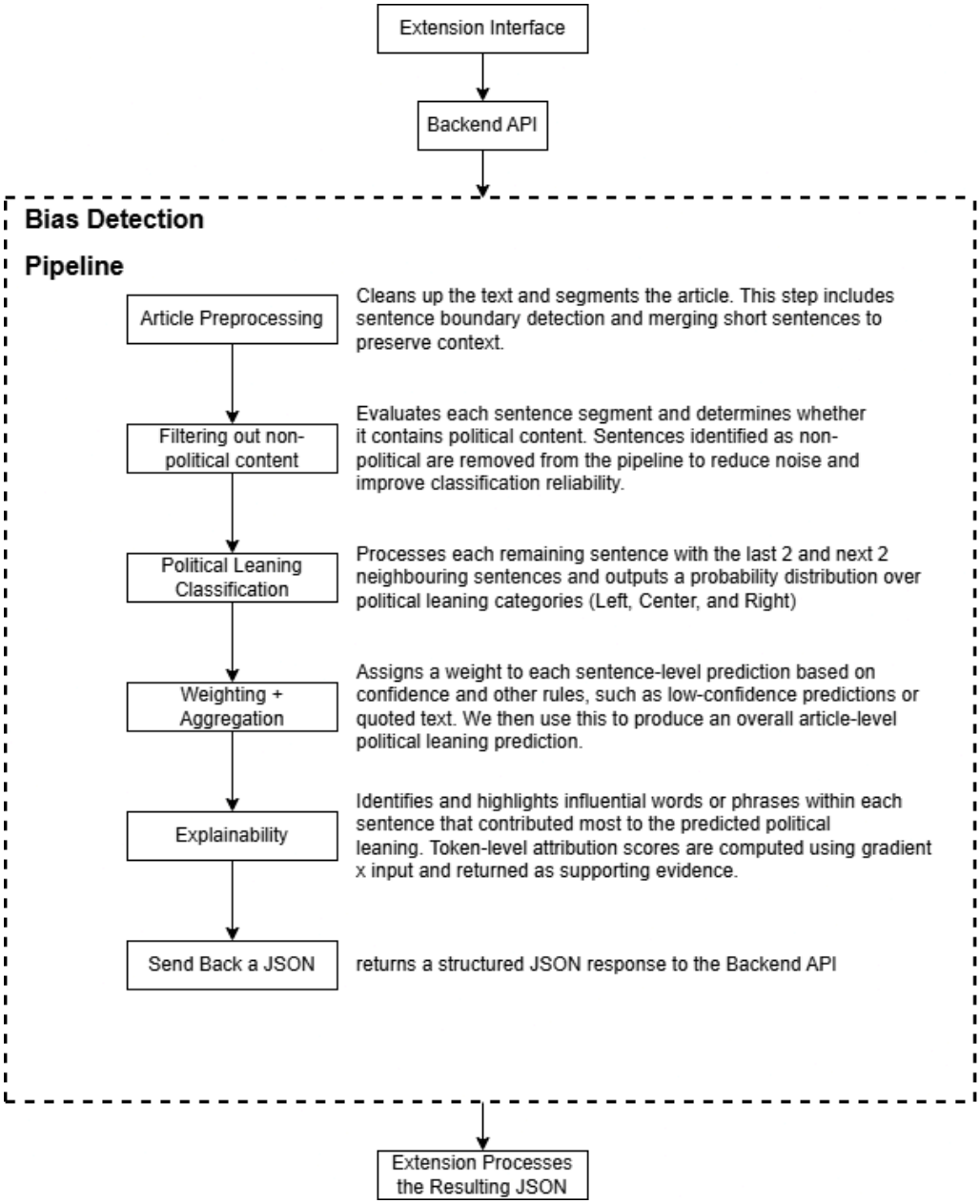


On Averitec, which had more far lesser but more sophisticated claims and required higher quality evidence, our performance with the retrieved evidence is consistent with the evaluation metrics of running the veracity classifier using the gold evidence from the dataset, with a slight F1 improvement on NEI.

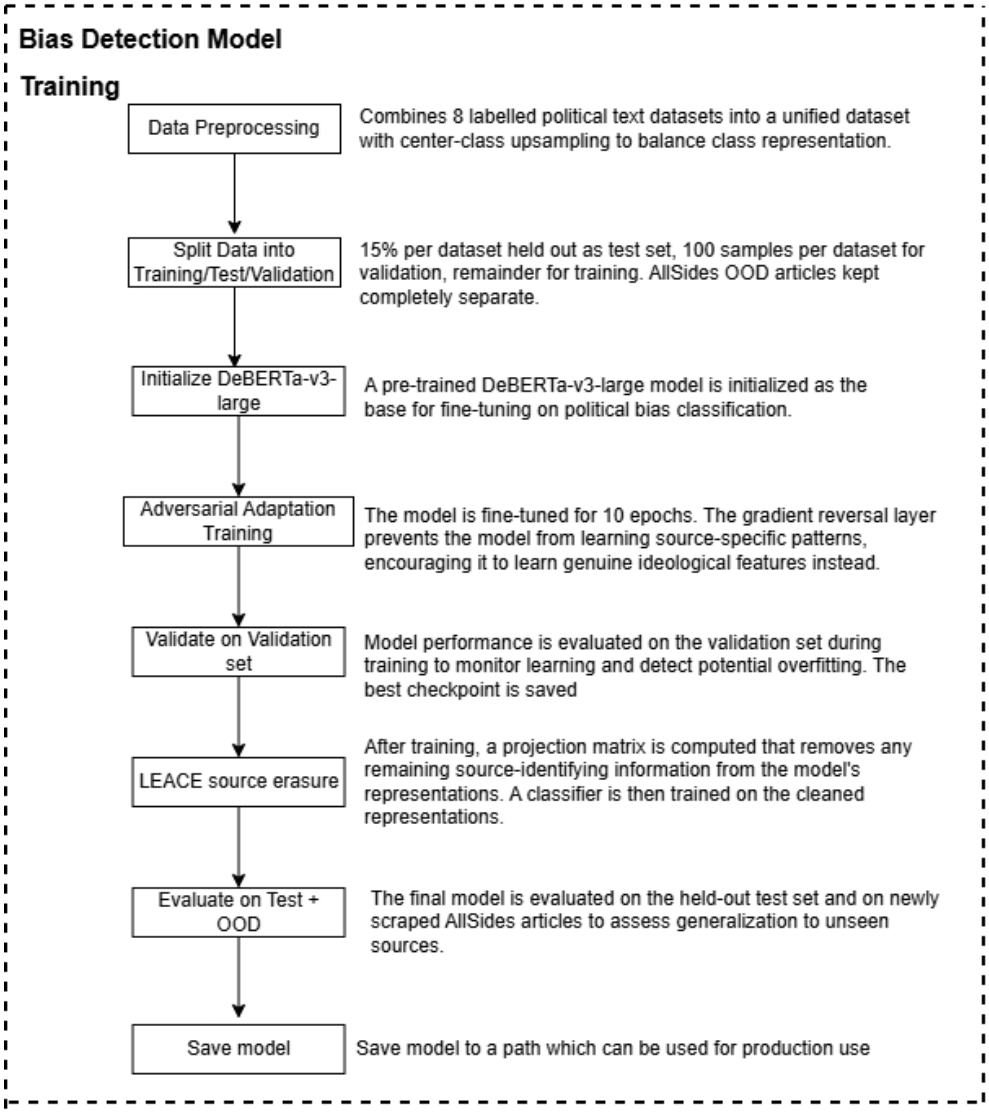
Model	Accuracy	Macro F1
Shared multi head attention model (with retrieved evidence)	0.67	0.51
Shared multi head attention model (with gold evidence)	0.73	0.52
Baseline model (trained and evaluated on Averitec)	0.6	0.35

Appendix

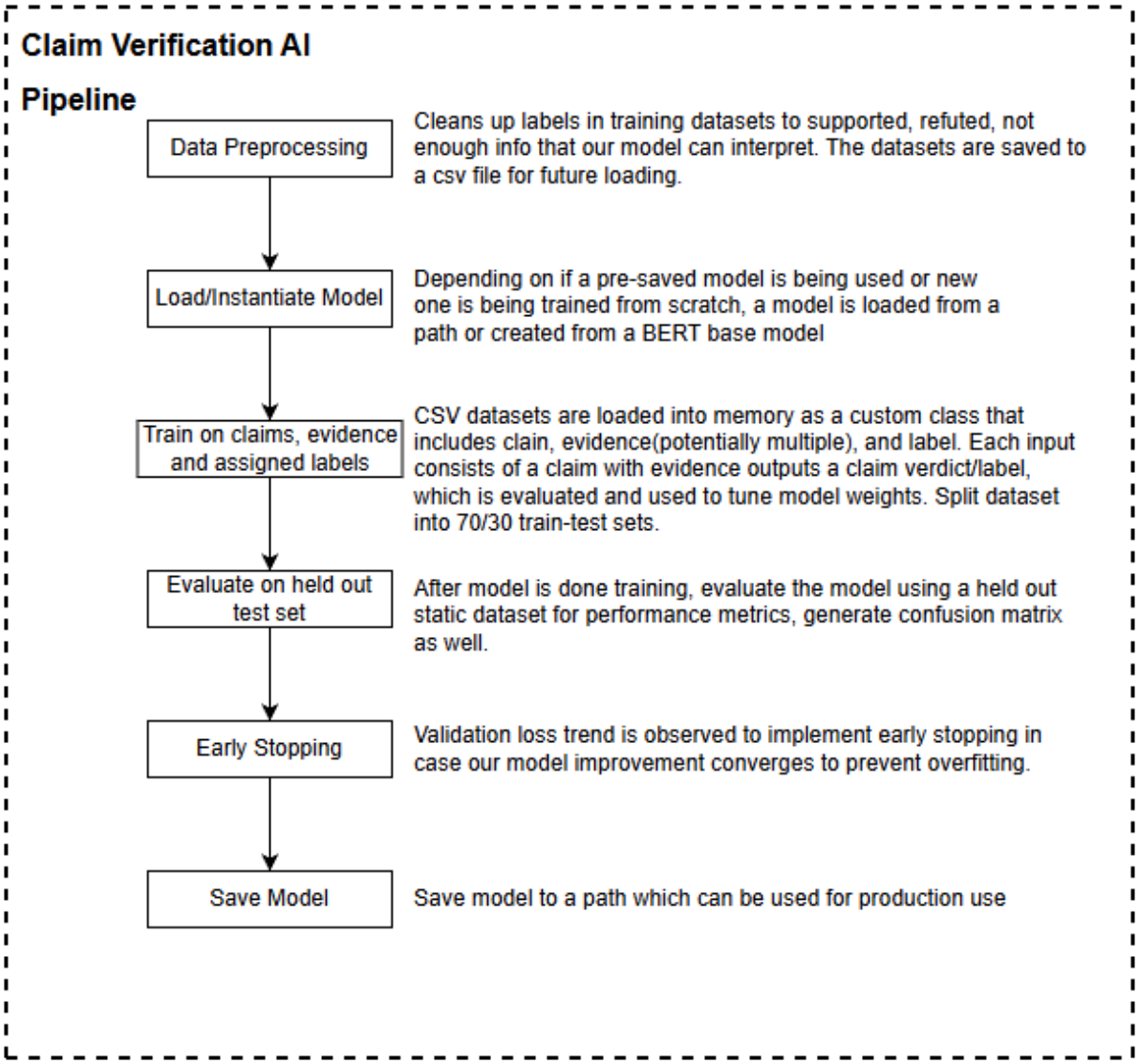
A. Bias Detection Diagram



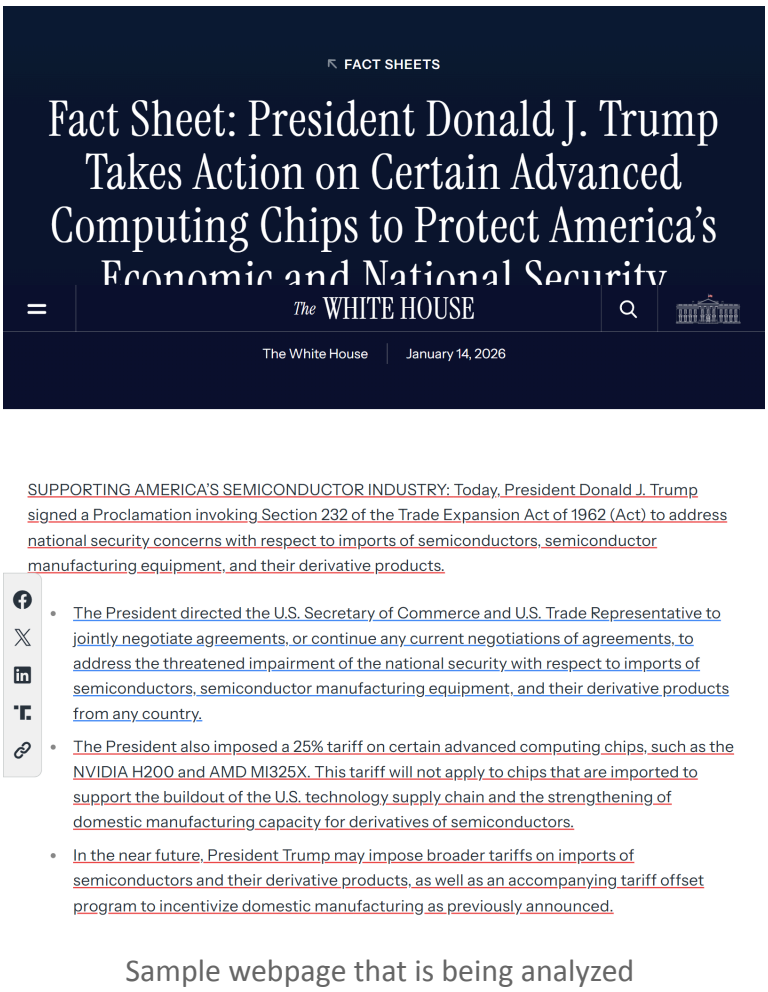
B. Bias Detection Training Diagram



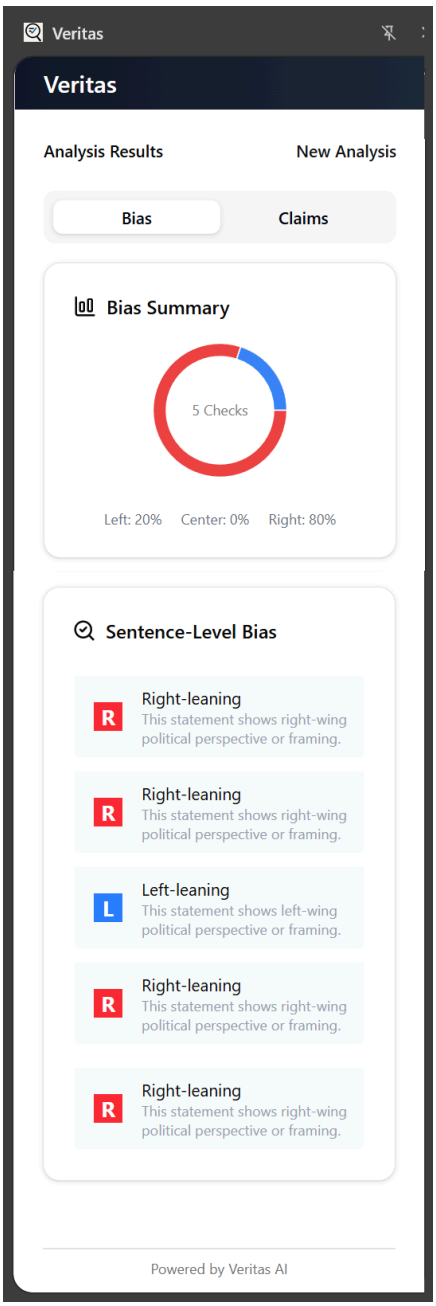
C. Claim Verification Diagram



D. Current Extension Interface



Sample webpage that is being analyzed



Extension side panel displaying results of the analysis

E. Figma Link

[Figma Design Link]